Session 2: CSS & Designing Your First Webpage

1. Session Overview & Objectives

Welcome to Session 2! You've already taken a massive step by building your first HTML webpage and setting up your professional developer environment. It is completely normal if looking at raw code still feels a bit intimidating—every software engineer starts exactly where you are, and learning to code is all about small, progressive technical wins. Today, we move from structure to style. We are going to transform your plain text webpage into a vibrant, beautifully designed site, all while continuing to build your confidence as a developer.

What We Will Learn:

- **CSS (Cascading Style Sheets):** If HTML is the skeleton of a house (the walls and roof), CSS is the interior design (the paint, furniture, and lighting). You will learn how to write CSS to add color, layout, and style to your webpage.
- **Targeting Elements:** We will learn how to use "selectors," "classes," and "IDs" to tell the computer exactly *which* part of the webpage you want to style.
- **The CSS Box Model:** A fundamental design rule that explains how spacing works on the internet (Margins, Borders, and Padding).
- **Professional Version History:** We will continue practicing our GitHub workflow. Think of a "commit" like saving a milestone in a video game—it allows you to safely track your project's history step-by-step so you never lose your hard work.

2. Conceptual Deep Dive

Separating HTML and CSS Professional developers keep their structure and style in separate files to keep things organized. Your HTML lives in index.html, and your design rules will live in a new file called style.css. We connect them using a simple <link> tag.

CSS Syntax: The Rulebook

CSS is written in "rules." A rule has two main parts: the **Selector** (what you want to style) and the **Declaration** (how you want it to look).

Below is the CSS File:

```
h1 {
  color: blue;
  font-size: 24px;
}
```

- h1 is the **selector**.
- color: blue; is the **declaration**. Always end declarations with a semicolon ;

Classes and IDs

Sometimes you have three buttons, but you only want one to be red.

- **Class (.classname):** Like a school uniform. Many elements can wear the same class. (e.g., <p class="highlight">). In CSS, you target it with a dot: .highlight { color: yellow; }
- **ID (#idname):** Like a Maldivian national ID card. It is unique and can only be given to *one* element on the page. (e.g., <nav id="main-menu">). In CSS, you target it with a hashtag: #main-menu { background: black; }

The CSS Box Model (Margin, Padding, Border) Every single thing on a webpage is actually a hidden rectangular box.

- **Real-world Analogy:** Imagine building a traditional coral stone house in the Maldives.
  - **Content:** The inside of the house itself.
  - **Padding:** The space between the inner walls and your furniture.
  - **Border:** The physical coral stone wall surrounding the house.
  - **Margin:** The open courtyard or street space outside the wall, keeping your house separated from your neighbor's house.

Brief Knowledge Check:

1. If I want to change the text color of *every* paragraph on my site, should I use a tag selector, a class, or an ID?
   Answer: class, to reduce the amount of code
2. Which part of the Box Model creates space *outside* the border of an element?
   Answer: Margin

3. Tools & GitHub Workflow

Tool Setup: Replit / VS Code & Developer Tools

1. Open your workspace (Replit or VS Code) from Session 1.
2. In your file explorer on the left, create a new file and name it exactly style.css.
3. Open index.html. Inside the <head> section, add this line to connect them: <link rel="stylesheet" href="style.css">
4. **Browser Developer Tools:** When viewing your live website, right-click any text and select "Inspect." This opens a panel where you can temporarily experiment with colors and sizes without breaking your real code.

The Git/GitHub Focus: The Everyday Workflow GitHub is a recurring professional practice, not an afterthought. Whenever you finish a small task (like changing background colors), you must save it to your history. In your terminal, use this exact, everyday cycle:

1. git status (Check what files you changed)
2. git add . (Stage all the new changes to be saved)
3. git commit -m "Add CSS styling" (Create a save point with a meaningful message)
4. git push (Upload your save point safely to the cloud)

AI Integration Note: If you choose to use an AI assistant like ChatGPT or GitHub Copilot for color palette ideas or CSS layout suggestions, remember that AI is a collaborative assistant. Do not blindly copy-paste code. You must be able to explain any code you commit to your p roject.

4. Practical Activity (Step-by-Step Walkthrough)

Step 1: Link Your Files & Test

- **Code Along:** Add the <link> tag to your HTML. Let's test it by making the whole page background soft blue.
- In style.css, write:

```
body {
  background-color: #e0f7fa;
  font-family: Arial, sans-serif;
}
```

- **Independent Practice:** Save and refresh your preview. Did the color change? If not, check your spelling of style.css in the link tag.

Step 2: Styling Text and Colors

- **Independent Practice:** Target your <h1> and <h2> tags. Give them a new color (try a hex code like #0077b6) and change the text-align to center.

Step 3: Applying the Box Model with Classes

- **Code Along:** Let's create a "card" look for your introduction section. Go to index.html and wrap your intro text in a tag with a class: .
- In style.css, add the Box Model properties:

```
.profile-card {
  background-color: white;
  padding: 20px;
  border: 2px solid #0077b6;
  border-radius: 10px;
  margin: 20px;
  width: 50%;
}
```

- **Independent Practice:** Adjust the padding and margin. See how the space inside and outside the box expands and shrinks.

Step 4: Save, Test, Commit, Push

- **Independent Practice:** Open your terminal. Run the four everyday Git commands listed in Section 3. Use a meaningful commit message like "Add profile card styling".

Debugging Tips:

- Errors are diagnostic tools, not personal failures.
- If CSS isn't applying, check for the three most common mistakes:

1. Forgetting the semicolon ; at the end of a line.
2. Missing a closing curly brace }.
3. Forgetting to save the file before refreshing the browser.

5. Answer Key & Expected Output

Expected Visual Output:

Your plain text page should now look like a structured profile. The background should be a soft color, the main headings centered, and your introduction paragraph enclosed in a neat, white box with a colored border and rounded corners.

Expected Terminal Output (Git):

Bash:

```
$ git commit -m "Add CSS styling"
[main 1a2b3c4] Add CSS styling
 2 files changed, 15 insertions(+)
 create mode 100644 style.css
$ git push
```

Solution Code (style.css):

```
/* Body styling affects the whole page */
body {
  background-color: #e0f7fa;
  font-family: 'Helvetica Neue', Arial, sans-serif;
  color: #333333;
}

/* Heading styling */
h1 {
  color: #0077b6;
  text-align: center;
}

/* Box Model applied to a specific class */
.profile-card {
  background-color: white;
  padding: 20px; /* Space inside the border */
  border: 2px solid #0077b6; /* The physical line */
  border-radius: 10px; /* Rounded corners */
  margin: 20px auto; /* Space outside, 'auto' centers the box */
  width: 60%; /* Restricts how wide the box can get */
}

/* Image styling */
img {
  width: 150px;
  border-radius: 50%; /* Makes a square image a perfect circle */
  display: block;
  margin: 0 auto; /* Centers the image */
}
```

6. Deliverables Checklist

Before wrapping up the session, ensure you can check off the following:

- A visual, styled webpage demonstrating changes from Session 1.
- A style.css file successfully created and linked to your HTML.
- CSS code that correctly utilizes selectors, colors, margin, padding, and borders.
- At least two meaningful GitHub commits pushed to your repository (e.g., "Add homepage structure", "Add CSS styling").
- A successful git push verified by checking your remote GitHub repository online.